

Generating Invoice Statements with DevExpress Reports

Tick quotes (select-all in the header), then either **Download** the invoice PDFs straight to the browser (just the quotes, no folders) or **Generate** them server-side into `Invoices/DD-MM-YYYY/<quote number>.pdf`, overwriting any existing file so there are no duplicates. A warning shows if nothing is selected.

0. What you're building & the flow

The selection and the report are shared; the two *outcomes* are separate functions:



Two buttons, two server actions, **one shared report-building helper**:

- **Download Statement** → returns the invoice PDFs to the browser. One quote = one PDF; several = a flat `.zip` of PDFs (no folder structure, just the quotes).
- **Generate Statement** → saves the PDFs on the server under `Invoices/DD-MM-YYYY/<quote number>.pdf`, overwriting an existing file for that quote so the folder never holds duplicates.

Prerequisites — DevExpress is a licensed product

DevExpress Reporting is **not** free or on nuget.org. You need an active DevExpress subscription, its NuGet feed configured in Visual Studio, and the `DevExpress.Reporting.Core` package. Exact class names / designer menus vary by version (v23.x, v24.x, v26.x...) — the *shape* of the code is stable; adjust version-specific bits to yours. This project uses `DevExpress.Reporting.Core 26.1.3`.

PART 1 — The checkbox column & the two buttons (the view)

1.1 — Add the checkbox column as the FIRST column

Views/Quote/Index.cshtml (table header – first <th>)

```

<thead class="table-dark">
  <tr>
    <th style="width:34px;">
      <input type="checkbox" id="selectAll" onclick="toggleSelectAll(this)" />
    </th>
    <th>QuoteId</th>
    ... your other headers ... <th>Actions</th>
  </tr>
</thead>

```

```

    </tr>
</thead>

```

Views/Quote/Index.cshtml (each row – first <td> inside the @foreach)

```

<tr data-id="@q.Id">
    <td><input type="checkbox" class="rowCheck" value="@q.Id" /></td>
    <td>@q.Id</td>
    ... your other cells ...
</tr>

```

Critical: your DataTables column indexes all shift by +1

A new column at position 0 pushes every existing column right by one. Your config references columns by index — QuotId was 0, Actions was 8; they're now 1 and 9. Update the options or the table breaks.

Views/Quote/Index.cshtml (DataTable options)

```

order: [[1, "desc"]],          // QuoteId is now column 1 (was 0)
columnDefs: [
    { orderable: false, searchable: false, targets: 0 }, // checkbox column
    { visible: false, targets: 1 },                      // QuoteId (hidden), was 0
    { orderable: false, targets: 9 }                    // Actions, was 8
]

```

1.2 — The two buttons and their scripts

Views/Quote/Index.cshtml (top row, next to the heading / Export buttons)

```

<button class="btn btn-outline-primary" onclick="downloadStatement()">Download Statement</button>
<button class="btn btn-primary" onclick="generateStatement()">Generate Statement</button>

```

Views/Quote/Index.cshtml (page <script>)

```

// header checkbox → tick/untick every visible row checkbox
function toggleSelectAll(source) {
    document.querySelectorAll(".rowCheck").forEach(cb => cb.checked = source.checked);
}

// the ticked QuoteIds – used by both buttons
function getSelectedIds() {
    return Array.from(document.querySelectorAll(".rowCheck:checked"))
        .map(cb => parseInt(cb.value));
}

function antiForgeryToken() {
    return document.querySelector('input[name="__RequestVerificationToken"]').value;
}

// ---- DOWNLOAD: get the PDFs back in the browser ----

```

```

function downloadStatement() {
    const ids = getSelectedIds();
    if (ids.length === 0) { alert("Please select at least one quote."); return; }

    fetch("/Quote/downloadstatement", {
        method: "POST",
        headers: { "Content-Type": "application/json", "RequestVerificationToken": antiForgeryToken() },
        body: JSON.stringify(ids)
    })
    .then(async res => {
        if (!res.ok) throw new Error(await res.text());

        // turn the file response into a browser download
        const blob = await res.blob();
        const dispo = res.headers.get("Content-Disposition") || "";
        const match = dispo.match(/filename="?(^)"?/);
        const name = match ? match[1] : "statement.pdf";

        const url = URL.createObjectURL(blob);
        const a = document.createElement("a");
        a.href = url; a.download = name; a.click();
        URL.revokeObjectURL(url);
    })
    .catch(err => alert(err.message));
}

// ---- GENERATE: save the PDFs on the server ----
function generateStatement() {
    const ids = getSelectedIds();
    if (ids.length === 0) { alert("Please select at least one quote."); return; }

    fetch("/Quote/generatestatement", {
        method: "POST",
        headers: { "Content-Type": "application/json", "RequestVerificationToken": antiForgeryToken() },
        body: JSON.stringify(ids)
    })
    .then(async res => {
        if (!res.ok) throw new Error(await res.text());
        alert(await res.text());
    })
    .catch(err => alert(err.message));
}

```

Two notes on selection

Paging: DataTables keeps only the current page's rows in the DOM, so the checkboxes cover the *current page*. For selecting across pages, use the DataTables API (`table.rows().nodes()`). **Token:** your layout already renders `@Html.AntiForgeryToken()` for logged-in users, so the hidden input exists — both functions just read it. Keep `[ValidateAntiForgeryToken]` on the actions.

PART 2 — The DevExpress quote report (QuoteReport)

2.1 — Install the reporting package

Package Manager Console (DevExpress feed configured)

```
dotnet add package DevExpress.Reporting.Core
```

The `.Core` package is the server-side, no-UI build — exactly what you need to render and export a report from a controller.

2.2 — The report model that feeds the report

One object carries the whole statement: the **header fields at the top level** and the **lines nested inside**. The report binds to this type through an `ObjectDataSource`, so the designer's Field List knows the shape at design time and the controller hands over everything in a single assignment at runtime:

Models/QuoteReportModel.cs (new file)

```
namespace QuotesProject.Models
{
    public class QuoteReportModel
    {
        public string? QuoteNumber { get; set; }
        public string Customer { get; set; } = "";
        public string? Address { get; set; }
        public string? ExternalOrderNumber { get; set; }
        public DateTime? OrderDate { get; set; }
        public DateTime? DueDate { get; set; }
        public List<QuoteLine> Lines { get; set; } = new();
    }
}
```

Why one model with nested lines (not report parameters)

An `ObjectDataSource` bound to this type gives the designer a real schema: the header fields bind like normal fields (they appear once per document), and the nested `Lines` collection drives a *nested detail report band* that repeats once per line. Everything arrives through one `DataSource` assignment at runtime — there is no list of report parameters to keep in sync with the model as it grows. Need `InvoiceDate` on the printout later? Add the property here and to the mapping in 3.2; the designer picks it up on the next build.

2.3 — Design the report (the designer)

1. **Add New Item** → **DevExpress Report**, choose *Blank*, in the `Reports` folder, named `QuoteReport`. This creates `QuoteReport : XtraReport` plus a designer surface (`.Designer.cs` + `.resx`).

2. **Bind the schema.** Build the project first (the wizard reflects over the compiled assembly). Then: report smart tag → **Data Source** → **Add Report Data Source...** → **Object** → pick the `QuotesProject` assembly → select `QuoteReportModel` → choose **Retrieve the data source schema** (schema only — no real data at design time). The Field List now shows the header fields plus a `Lines` node.
3. **Header** → **PageHeader**. Drag `QuoteNumber`, `Customer`, `Address`, `OrderDate`, `DueDate` into the PageHeader band, plus static labels ("Quote #:", "Customer:", ...) and your company details.
4. **Lines** → **a nested detail report**. Right-click the design surface → **Insert Detail Report** → **"Lines"**. This adds a nested band whose `DataMember` is `Lines` — it repeats once per quote line. Drag `Item`, `Quantity`, `Price`, `Discount` into its Detail band (or use an `XRTTable` for clean columns), and add a **Group Header** band to it for the column captions.
5. **Totals**. Per-line total: a label in the Lines detail with the expression `[Quantity] * [Price] * (1 - [Discount] / 100)`. Grand total: a **Report Footer** band on the nested report with `sumSum([Quantity] * [Price] * (1 - [Discount] / 100))`. (Adjust if `Discount` is an amount, not a percent.)
6. **Formatting**. Set `TextFormatString` per control — `{0:yyyy/MM/dd}` for dates, `{0:n2}` for money; right-align the numeric columns. An `XRPageInfo` in the BottomMargin gives page numbers.

Schema at design time, data at runtime

The wizard stores only the *type* — the generated designer code says `objectDataSource1.DataSource = typeof(QuoteReportModel)` and wires `this.DataSource = objectDataSource1`. No database is touched while designing; the real, filled model is pushed into that same component by the controller helper in 3.2.

PART 3 — The controller: one helper, two actions

3.1 — Inject what the actions need

Controllers/QuoteController.cs

```
private readonly IQuoteService _quoteService;
private readonly IQuoteLineService _quoteLineService; // each quote's lines
private readonly IWebHostEnvironment _env;           // for the server-save path

public QuoteController(IQuoteService quoteService,
                      IQuoteLineService quoteLineService,
                      IWebHostEnvironment env)
{
    _quoteService = quoteService;
    _quoteLineService = quoteLineService;
    _env = env;
}
```

3.2 — The shared helper: build one invoice PDF

Both actions build the PDF the same way, so that lives in one private helper. It returns the file name (`<quote number>.pdf`) and the PDF bytes — the actions differ only in *where the bytes go*.

Controllers/QuoteController.cs (private helper)

```
using DevExpress.DataAccess.ObjectBinding;
using QuotesProject.Reports;

private (string fileName, byte[] pdf) BuildStatement(int quoteId)
{
    Quote quote = _quoteService.GetQuoteById(quoteId);
    List<QuoteLine> lines = _quoteLineService.GetQuoteLines(quoteId);

    var model = new QuoteReportModel // header + lines in one object
    {
        QuoteNumber      = quote.QuoteNumber,
        Customer          = quote.Customer,
        Address           = quote.Address,
        ExternalOrderNumber = quote.ExternalOrderNumber,
        OrderDate         = quote.OrderDate,
        DueDate           = quote.DueDate,
        Lines             = lines
    };

    using var report = new QuoteReport();

    // push the data into the designer's ObjectDataSource – the report AND the
    // nested Lines detail report both resolve through this one component
    var ods = report.ComponentStorage.OfType<ObjectDataSource>().First();
    ods.DataSource = new List<QuoteReportModel> { model };

    report.CreateDocument(); // render

    using var ms = new MemoryStream();
    report.ExportToPdf(ms); // PDF → bytes

    return ($"{quote.QuoteNumber}.pdf", ms.ToArray());
}
```

Why set the ObjectDataSource, not report.DataSource

The designer wired `report.DataSource → objectDataSource1`, and the nested Lines detail report resolves its `Lines` member through that same component. Filling the `ObjectDataSource` (found via `ComponentStorage`) keeps every binding — header fields and lines — working. Assigning `report.DataSource` directly would bypass the component the bands point at. The report is bound to a *list* with one `QuoteReportModel`: one statement per document.

Why a helper here

You asked to keep helpers minimal — this one earns its place because *both* actions need the identical "load data → fill model → bind report → render → bytes" sequence. Writing it once means the download and the save can never drift apart, and each action is then just three or four lines about its destination.

3.3 — Action A: DOWNLOAD (no folders, just the quotes)

Controllers/QuoteController.cs

```
using System.IO.Compression;
using Microsoft.AspNetCore.Authorization;

[HttpPost("downloadstatement")]
[Authorize]
[ValidateAntiForgeryToken]
public IActionResult DownloadStatement([FromBody] int[] quoteIds)
{
    if (quoteIds == null || quoteIds.Length == 0)
        return BadRequest("No quotes selected.");

    // one quote → return that single PDF
    if (quoteIds.Length == 1)
    {
        var (fileName, pdf) = BuildStatement(quoteIds[0]);
        return File(pdf, "application/pdf", fileName);
    }

    // several quotes → a FLAT zip: just the PDFs at the root, no folders
    using var zipStream = new MemoryStream();
    using (var archive = new ZipArchive(zipStream, ZipArchiveMode.Create, leaveOpen: true))
    {
        foreach (int id in quoteIds)
        {
            var (fileName, pdf) = BuildStatement(id);
            var entry = archive.CreateEntry(fileName); // file name only → no folder path
            using var entryStream = entry.Open();
            entryStream.Write(pdf, 0, pdf.Length);
        }
    }

    return File(zipStream.ToArray(), "application/zip", "Statements.zip");
}
```

How it decides: a browser response carries one file, so a single selected quote comes back as its own `<number>.pdf` ; multiple come back as `Statements.zip` containing the PDFs *flat* (`CreateEntry(fileName)` with no path = no folders). The `File(...)` result sets `Content-Disposition` , which the download script in Part 1 reads for the file name.

3.4 — Action B: GENERATE (server-side save, overwrite → no duplicates)

Controllers/QuoteController.cs

```
[HttpPost("generatestatement")]
[Authorize]
[ValidateAntiForgeryToken]
public IActionResult GenerateStatement([FromBody] int[] quoteIds)
{
    if (quoteIds == null || quoteIds.Length == 0)
        return BadRequest("No quotes selected.");

    // Invoices/DD-MM-YYYY (today; folder created if missing)
    string dateFolder = DateTime.Now.ToString("dd-MM-yyyy");
    string folder = Path.Combine(_env.ContentRootPath, "Invoices", dateFolder);
    Directory.CreateDirectory(folder);

    foreach (int id in quoteIds)
    {
        var (fileName, pdf) = BuildStatement(id);
        string path = Path.Combine(folder, fileName);

        // WriteAllBytes REPLACES the file if it already exists (and creates it if not),
        // so re-generating the same quote in this dated folder overwrites – no duplicates.
        System.IO.File.WriteAllBytes(path, pdf);
    }

    return Ok($"{quoteIds.Length} statement(s) saved to Invoices/{dateFolder} (existing files overwritten).");
}
```

How "no duplicates" works

The file path is `Invoices/<today>/<quote number>.pdf`, and `File.WriteAllBytes` overwrites whatever is already at that path. So generating quote `Q1001` twice today produces *one* file, not two — the second run replaces the first. (Each day gets its own dated folder, so you keep one copy *per day* as a history; drop the `dateFolder` segment if you want a single copy ever.)

3.5 — Path & naming notes

- **Where files land:** `_env.ContentRootPath` is your project root in development, so saves appear under `QuotesProject/Invoices/DD-MM-YYYY/`. Swap it for any base path you prefer.
- **Date:** `DateTime.Now.ToString("dd-MM-yyyy")` → e.g. `22-07-2026`, always today.
- **File-name safety:** if a quote number can contain `/ \ : *` etc., strip them before using it as a file name (illegal in file paths and zip entries).
- **Download vs save are independent:** Download never writes to disk; Generate never sends anything to the browser. Same helper, different destination.

Step 4 — Test it

Do this	You should see
1. Either button with nothing ticked	Warning alert; no request sent.
2. Tick <i>one</i> row → Download	A single <code><number>.pdf</code> downloads.
3. Tick <i>two</i> rows → Download	<code>Statements.zip</code> downloads; inside are two PDFs, flat, no folders.
4. Tick two rows → Generate	"2 statement(s) saved..."; two PDFs under <code>Invoices/<today>/</code> .
5. Generate the same quote again	Still one file for it in today's folder — overwritten, not duplicated.
6. Header select-all → Generate	A PDF per quote on the current page.
7. POST an empty array (dev-tools) to either action	400 "No quotes selected" — server guard holds.
8. Confirm the grid still sorts/searches	Quoteld hidden, sort works → your <code>columnDefs</code> shift (1.1) is correct.
9. Open a generated PDF	Header fields filled in once; one row per quote line in the nested detail; totals correct.

5. Recap & gotcha checklist

Thing	Watch out for
Two functions, one helper	<code>BuildStatement</code> makes the PDF bytes; <code>DownloadStatement</code> returns them, <code>GenerateStatement</code> saves them.
Download shape	1 quote → a PDF; many → a flat zip (no folders).
Server save + overwrite	<code>File.WriteAllBytes</code> to <code>Invoices/date/number.pdf</code> replaces existing → no duplicates per day.
DataTables indexes	Checkbox at column 0 shifts every index +1 — update <code>order</code> + <code>columnDefs</code> .
Empty selection	Warn on the client <i>and</i> 400 on the server.
DevExpress licence	Required; packages from the DevExpress feed, not nuget.org.
Model binding	One <code>QuoteReportModel</code> through the <code>ObjectDataSource</code> — header fields bind once, <code>Lines</code> repeats in the nested detail report. Fill the data source via <code>ComponentStorage</code> , not <code>report.DataSource</code> .
Designer schema	Build the project before running the Data Source Wizard; it stores <code>typeof(QuoteReportModel)</code> , schema only.
File-name safety	Strip illegal characters from the quote number.

Done

A checkbox grid with select-all, a warning on empty selection, and *two* outcomes from one shared report helper: **download** the invoice PDFs straight to the browser (flat, just the quotes) and **generate** them server-side into a dated folder that overwrites rather than duplicates. The report itself is `QuoteReport` bound to a `QuoteReportModel` via an `ObjectDataSource` — header once, lines in a nested detail report. The pattern — *collect ids → POST → per id build the PDF → send or save* — extends to any report you design next.